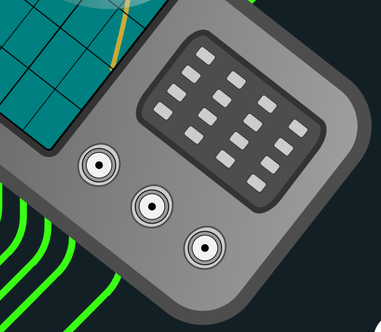
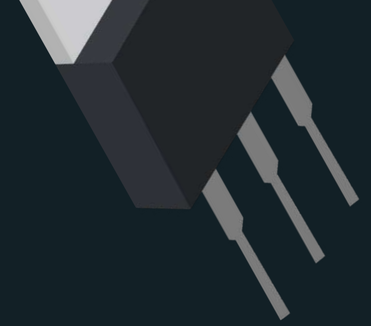

Emergency
Rescue Terminal
using Morse
Code

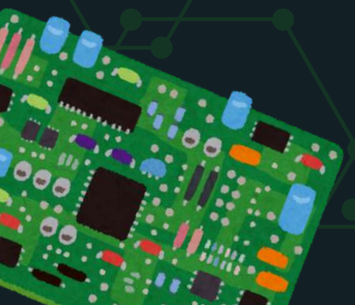
Our Team

- Gathin
- Giri Prasath
- Mithun Prasath
- Naren Karthick





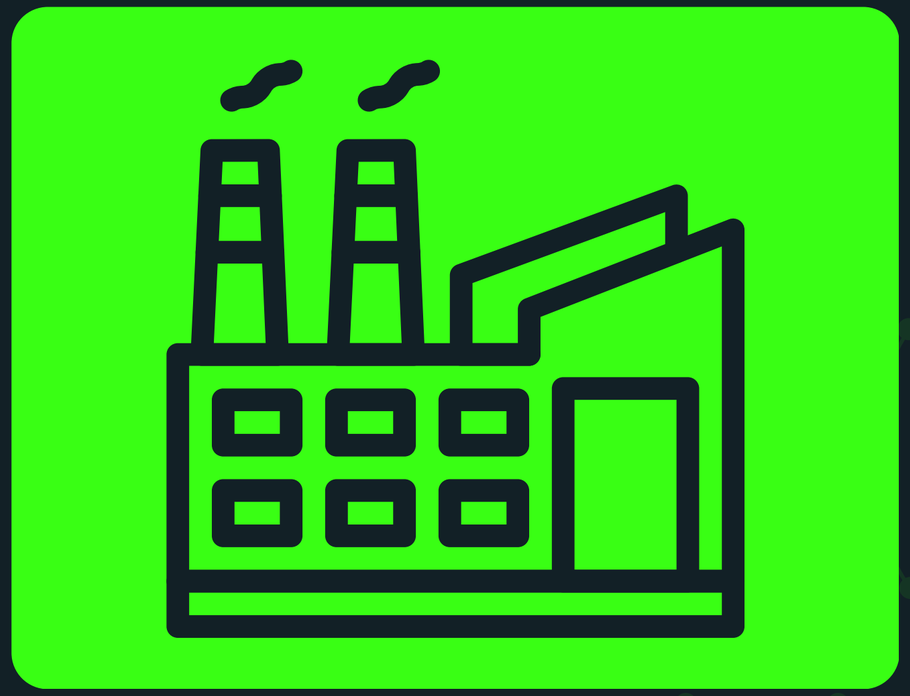




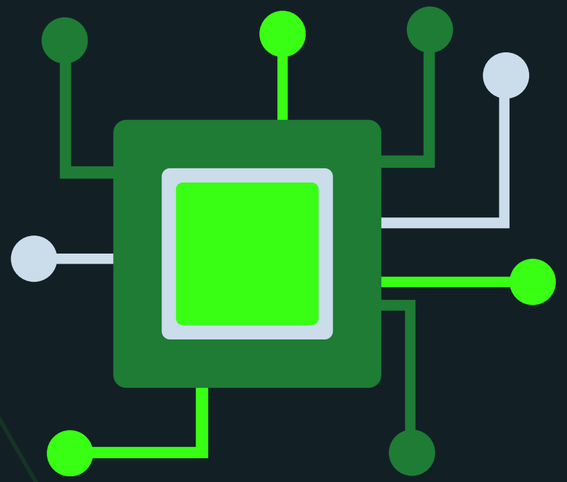


Problem Statement

Cellular networks fail in remote environments, and pulling out a glowing smartphone during a threat exposes a victim's location. This project addresses the critical need for a tactile, pocket-sized terminal capable of silently broadcasting GPS distress data through covert optical and audio Morse code.



Introduction



- Core Concept: A standalone, pocket-sized IoT emergency communication and tracking terminal.
- Primary Objective: Enables off-grid distress signaling in remote "dead zones" where cellular networks and smartphones fail completely.
- Covert Advantage: Engineered with Stealth Mode (zero sound, zero screen backlight) to transmit GPS coordinates safely via optical Morse code without alerting an active threat.
- Embedded Reliability: Built on robust Arduino architecture featuring a non-blocking state machine for instant, lag-free hardware response during high-stress emergencies

Components

- Microcontroller Core

Component: Arduino Uno

Purpose: Acts as the central processing unit. Executes the non-blocking Finite State Machine (FSM) firmware, coordinates real-time button polling, manages state transitions (STATE_OFF to STATE_PANIC_SOS), and coordinates serial telemetry.

- User Interface & Display

Component: 16x2 LCD Display (Pins 7, 6, 10, 11, 12, 13) + Backlight Control (Pin A0)

Purpose: Provides a visual user interface (UI) to display boot sequences, main menus, preset emergency options (SOS, INJURED, NEED WATER), and transmission status. Backlight pin is dynamically software-controlled to kill light output during Stealth Mode.

- Input Control Matrix (Tactical Push Buttons)

Component: 4x Tactile Push Buttons (Configured with Internal Pull-Ups)

ON / Power Button (Pin 2): Soft-boots the system from deep sleep; requires a 1-second continuous hold to trigger safe shutdown.

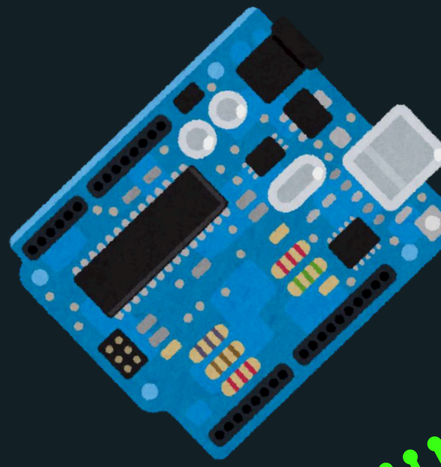
UP Navigation Button (Pin 3): Scrolls backward through menus; acts as part of the 3-second panic combination.

DOWN Navigation Button (Pin 4): Scrolls forward through menus; acts as part of the 3-second panic combination.

SELECT Confirmation Button (Pin 5): Confirms menu selections, initiates action states, and toggles flashlight mode.

Purpose: Allows blind, tactile, glove-friendly operation in high-stress, low-visibility environments without requiring a complex touchscreen.





4. Emergency Output Peripherals
Component: Piezo Buzzer (Pin 8)

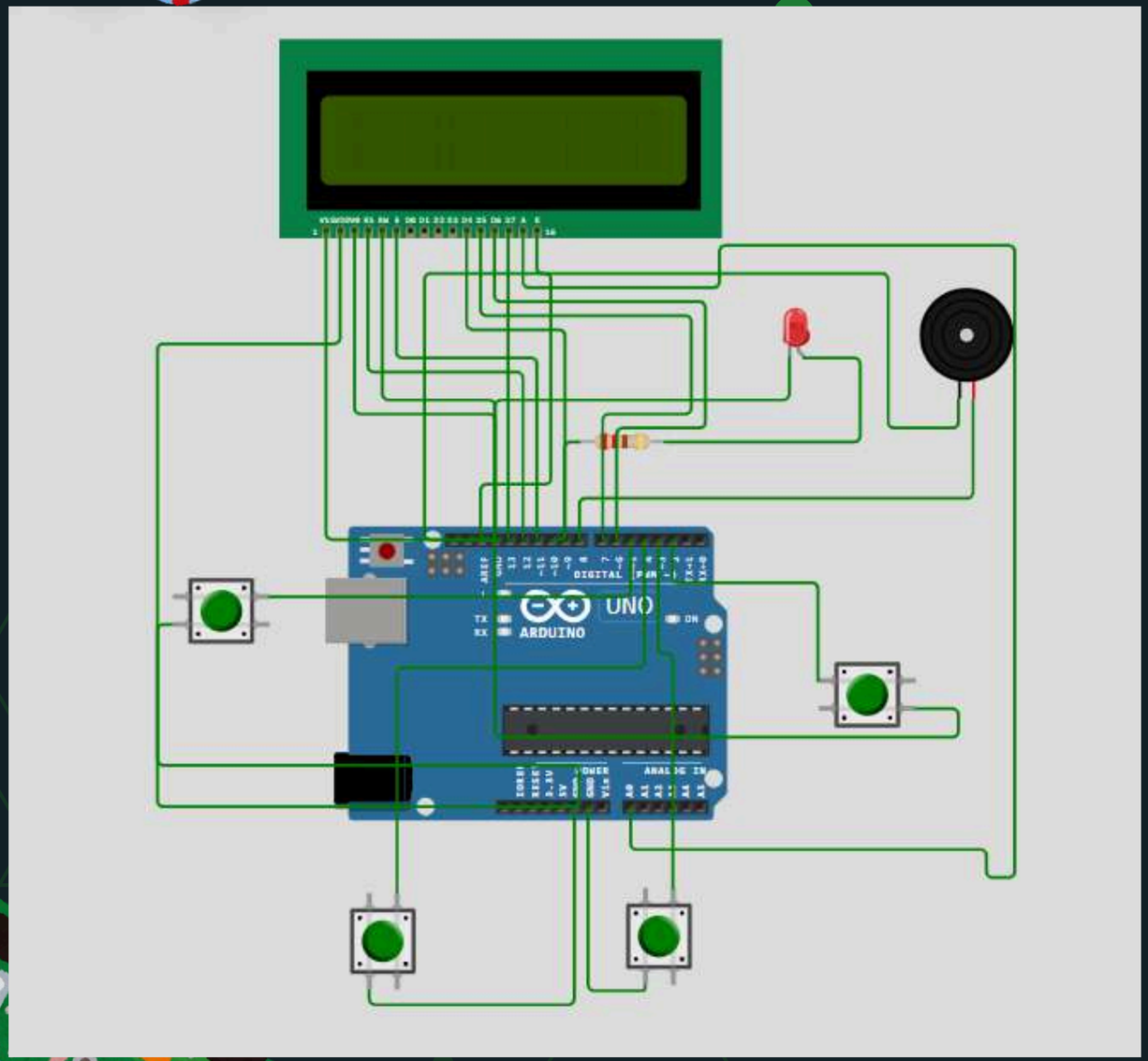
Purpose: Acoustic signal emitter driven by Arduino's tone() function to broadcast audible Morse code distress sequences. Automatically muted when Stealth Mode or Panic Mode is active.

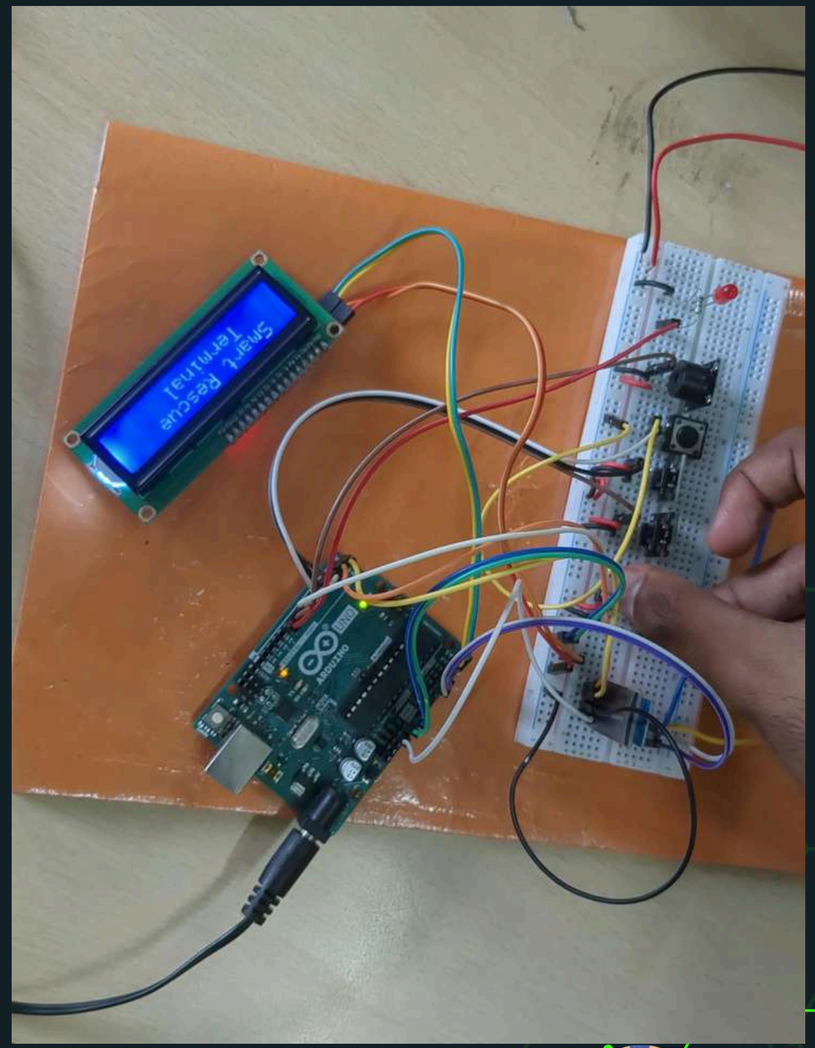
Component: High-Brightness Red LED (Pin 9) with 220Ω Current-Limiting Resistor

Purpose: Dual-function optical emitter. Serves as a manual flashlight torch and automatically flashes precise optical Morse code pulses. Remains the only active output during 100% invisible Stealth Mode.

5. Telemetry & Tracking Unit
Component: GPS Module

Purpose: Captures and caches geographic coordinates (17.38 78.48) to embed latitude and longitude data into transmitted emergency Morse messages.


UNO
ARDUINO
DIGITAL
ANALOG IN
POWER


Smart Rescue
Terminal

System Architecture & Workflow Flowchart

ATmega328P Finite State Machine (FSM) Execution Flow

Non-Blocking Firmware Core

STATE 01

System Boot

Device rests in **STATE_OFF** deep sleep. Tap Pin 2 ON button to initialize UI and LCD backlight.

STATE 02

Main Menu

Navigate modes using UP/DOWN buttons. Select SOS, predefined messages, flashlight, or toggle Stealth Mode.

STATE 03

GPS & Morse Engine

Retrieves cached location coordinates and converts text string into precise optical and acoustic pulse sequences.

Stealth Mode Protocol
Cuts LCD screen backlight (0% light) and mutes buzzer audio (0% sound). Transmits distress signals via silent optical LED flashing.

Panic SOS Hardware Override
Holding UP + DOWN buttons simultaneously for 3 seconds instantly forces a screen blackout and silent emergency broadcast.

System Architecture & Workflow Flowchart

Smart Morse Code Rescue Terminal - Operational State & Execution Path

1. System OFF
Deep Sleep (State: OFF)

→

2. Power ON
Tap Pin 2 (ON Button)

→

3. Welcome & Menu
LCD Init & Main Menu

↓ User Selection

4. GPS Cache
Retrieve Coords (17.38 78.48)

→

5. Morse Engine
Translate Text to Pulses

→

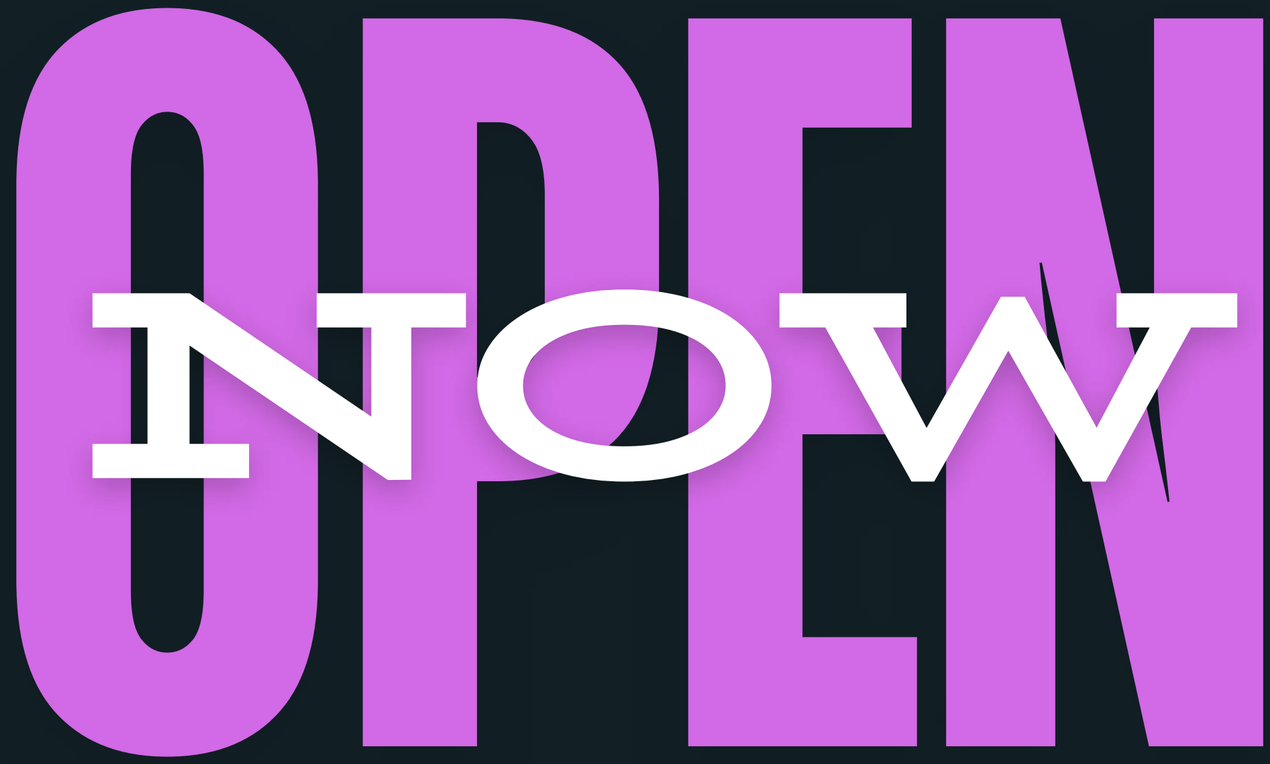
6. Hardware Output
LED (Pin 9) & Buzzer (Pin 8)

↓ Critical Safety Interrupts

Stealth Mode
LCD Backlight OFF / Muted Audio

Panic SOS Mode
Hold UP+DOWN (3s) → Dark Lockdown

Safe Shutdown
Hold ON Button (1s) → Power Cut



https://wokwi.com/projects/470988327414979585

Project Summary & Impact

- Core Achievement: Successfully designed, programmed, and validated a low-cost, highly reliable personal safety and emergency communication terminal bridging the gap between off-grid dead-zone tracking and covert distress signaling.

Engineering Milestones:

- Implemented a robust, non-blocking Finite State Machine (FSM) architecture ensuring zero input lag during high-stress execution.
- Successfully integrated critical anti-kidnapping safeguards: Stealth Mode (zero light, zero sound) and a 3-second Panic SOS lockdown trigger.
- Automated telemetry translation from cached GPS coordinates into precise optical and audio Morse code sequences.

Limitations Occured

This prototype (as captured in the current firmware and Wokwi circuit) has a few areas that are stubbed for simulation and should be addressed before field deployment:

1. GPS is not yet live. The TinyGPS++ library and SoftwareSerial on pins A1/A2 are included and initialized, but updateLocationCache() currently HARDCODES lastKnownLocation = "17.38 78.48" rather than parsing NMEA sentences from an actual GPS module. No GPS part is present in the current Wokwi circuit JSON either. This is the singlehighest-priority item for real-world development.

2. No persistent storage. Stealth Mode and menu state reset every time the device is powered off; there is no EEPROM/flash use to remember user .

3.No real RF/long-range transmission. The buzzer/LED is the entire "transmission" -- there's no radio, GSM, or LoRa module actually broadcasting the SOS beyond line-of-sight/earshot. This is intentional for the current prototype scope (a local, no-network distress beacon) but limits range.

4. No power/battery monitoring. There is no battery voltage sense or low-battery warning in the current firmware.

5. Panic Mode is unconditional. Once triggered, there's no cancel/undo -- this is a deliberate design trade-off for reliability under duress but is worth noting for a user manual.

Limitations Barriers yet

1. No persistent storage. Stealth Mode and menu state reset every time the device is powered off; there is no EEPROM/flash use to remember user .

2.No real RF/long-range transmission. The buzzer/LED is the entire "transmission" -- there's no radio, GSM, or LoRa module actually broadcasting the SOS beyond line-of-sight/earshot. This is intentional for the current prototype scope (a local, no-network distress beacon) but limits range.

Comparing us with Existing Model



Mc Murdo Fast Find



Thank You